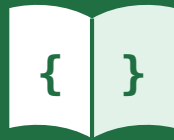

Modül 02 — Konfigürasyon ve Auto-Configuration

Ders Notları

Spring Boot Edu



2026-09-23

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Externalized Configuration	2
2.2 Relaxed Binding	2
2.3 Auto-Configuration	3
3. Adım Adım Örnekler	3
3.1 Ayar Kaynakları ve Öncelik Sırası	3
3.2 @ConfigurationProperties ile Tipli Ayarlar	5
3.3 @Value ile Tek Değer Enjeksiyonu	6
3.4 Profiller ve Profil Grupları	7
3.5 spring.config.import	8
3.6 Kendi Starter'ınızı Yazmak	9
3.7 Conditions Report ile Hata Ayıklama	10
4. Sık Yapılan Hatalar ve En İyi Pratikler	11
5. Özet	12
6. İleri Okuma	12

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Bir ayarın hangi kaynaktan geldiğini ve hangi kaynağın kazandığını açıklamak
- Ayarları `@ConfigurationProperties` record'larıyla tipli, doğrulanmış ve değişmez hale getirmek
- `@Value`'yu doğru yerde kullanmak ve sınırlarını bilmek
- Profilleri, profil gruplarını ve profile özel YAML belgelerini kullanmak
- `spring.config.import` ile ek konfigürasyon dosyaları yüklemek
- Auto-configuration'ın nasıl çalıştığını anlamak ve kendi starter'ınızı yazmak
- Conditions report ile bir auto-configuration'ın neden çalıştığını veya çalışmadığını bulmak

Ön koşullar: Modül 01 (IoC, @Bean, koşullu bean'ler) · **Tahmini süre:** 4 saat

2. Kavramlar

2.1 Externalized Configuration

Aynı uygulama (aynı JAR) geliştirici makinesinde, testte ve üretimde farklı ayarlarla çalışır. Spring Boot ayarları koddan ayırır ve pek çok kaynaktan okur: YAML/properties dosyaları, ortam değişkenleri, komut satırı argümanları ve daha fazlası.

Tüm kaynaklar Environment içinde **sıralı bir liste** olarak durur. Bir anahtar istendiğinde listede ilk bulunan değer kazanır. Önemli kaynaklar, **düşükten yükseğe** doğru:

1. application.yml (JAR içinde)
2. application-{profil}.yml (JAR içinde)
3. JAR dışındaki application.yml ve application-{profil}.yml
4. İşletim sistemi ortam değişkenleri
5. Java sistem özellikleri (-Dkey=value)
6. Komut satırı argümanları (--key=value)
7. Testlerde @SpringBootTest(properties = ...), @TestPropertySource

★ İpucu

Kural basittir: **daha dışarıdaki, daha özel olan kazanır**. Paketlenmiş varsayılanlar en zayıf, komut satırı en güçlüdür.

2.2 Relaxed Binding

Spring Boot anahtar adlarını esnek eşler. YAML'da support-email, Java'da supportEmail, ortam değişkeninde BOOKSTORE_STORE_SUPPORTEMAIL aynı ayardır. Ortam değişkeni için ku-

ral: noktalar _ olur, tireler silinir, harfler büyür.

2.3 Auto-Configuration

Spring Boot'un "sihri", sınıf yolundaki kütüphanelere bakıp gereken bean'leri sizin yerinize oluşturmasıdır. Her auto-configuration aslında sıradan bir @Configuration sınıfıdır, ama **koşullarla** korunur:

Koşul	Anlamı
@ConditionalOnClass	Belirtilen sınıf sınıf yolundaysa
@ConditionalOnMissingBean	Uygulama aynı tipte bir bean tanımlamadıysa ("geri çekilme", back-off)
@ConditionalOnProperty / @ConditionalOnBooleanProperty	Bir ayar belirli bir değerdeyse

Boot başlarken tüm jar'lardaki META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration dosyalarını okur ve orada listelenen sınıfları değerlendirir.

i Not

Spring Boot 4 modülerdir. Örneğin veritabanı auto-configuration'ları spring-boot-jdbc modülündedir. O modül sınıf yolunda yoksa ilgili auto-configuration'lar hiç **aday bile olmaz** (bölüm 3.7).

3. Adım Adım Örnekler

Bu modül, kendi starter'ımızı da derlediği için -am (also-make) ile çalıştırılır:

```
./mvnw -pl modules/02-configuration/lesson -am spring-boot:run
```

Örnekler com.springbootedu.configuration paketindedir. Starter ise modules/02-configuration/starter/ altındaki iki ayrı Maven modülüdür.

3.1 Ayar Kaynakları ve Öncelik Sırası

Amaç: Bir değer nereden geldiğini görmek ve kaynakların öncelik sırasını deneyerek öğrenmek.

Paketlenmiş varsayılanlar application.yaml dosyasındadır:

```
bookstore:
  store:
    name: Kitapçı
    support-email: destek@kitapci.example           # kebab-case in YAML → supportEmail
    ↪ in Java
    categories:
      - roman
      - bilim
      - yazılım
    shipping:
      free-from: "500.00"                         # quoted: unquoted 500.00 is a YAML
    ↪ float → 500.0
      delivery-time: 2d
```

PropertyOrigins, Environment'taki kaynakları öncelik sırasıyla gezer ve anahtarı ilk içeren kaynağın adını döndürür:

```
@Component
public class PropertyOrigins {

    private final ConfigurableEnvironment environment;

    public PropertyOrigins(ConfigurableEnvironment environment) {
        this.environment = environment;
    }

    public String sourceOf(String key) {
        for (PropertySource<?> source : environment.getPropertySources()) { // highest
            ↪ priority first
                if (source.getName().equals("configurationProperties")) {
                    continue; // Boot's combined view over all other
                        ↪ sources – skip it
                }
                if (source.containsProperty(key)) {
                    return source.getName();
                }
            }
        }
        return "(not set)";
    }
}
```

Çalıştırın: Aynı ayarı komut satırından ezin:

```
./mvnw -pl modules/02-configuration/lesson -am spring-boot:run \
  -Dspring-boot.run.arguments="--bookstore.store.name=Komut"
```

Beklenen çıktı:

```
== 3.1 Property sources
bookstore.store.name = Komut ← commandLineArgs
bookstore.store.support-email ← Config resource 'class path resource [application.yaml]'
↪ ...
```

Ortam değişkeniyle denemek için: `BOOKSTORE_STORE_NAME="Ortam" ./mvnw -pl modules/02-configuration/lesson -am spring-boot:run`

Testi: `sources/PropertySourcesTest` — komut satırının YAML'ı ezdiğini ve ortam değişkeni adının relaxed binding ile eşlendiğini doğrular.

3.2 @ConfigurationProperties ile Tipli Ayarlar

Amaç: İlgili ayarları tek bir değişmez record'da toplamak, dönüştürmek ve başlangıçta doğrulamak.

```
@ConfigurationProperties("bookstore.store")
@Validated // validate at startup – fail fast
public record StoreProperties(
    @NotBlank String name,
    @NotBlank @Email String supportEmail,
    @DefaultValue("TRY") Currency currency, // "TRY" → java.util.Currency,
    ↪ converted by Boot
    @DefaultValue List<String> categories, // empty list instead of null
    @Valid @NotNull Shipping shipping, // @Valid: validate the nested
    ↪ object as well
    @Nullable String banner) {

    /**
     * @param freeFrom orders from this amount ship for free
     * @param deliveryTime promised delivery time, e.g. 2d or 36h
     */
    public record Shipping(
        @NotNull @DecimalMin("0") BigDecimal freeFrom,
        @DefaultValue("3d") Duration deliveryTime) { // "2d", "36h", "PT12H" all work
    }
}
```

Boot, metinleri hedef tiplere kendisi dönüştürür: "2d" → Duration, "TRY" → Currency, YAML listesi → List<String>. @Validated sayesinde geçersiz bir ayar uygulamanın **başlamasını engeller**. Hata, üretimde bir isteğe kadar beklemes.

Beklenen çıktı:

```
== 3.2 @ConfigurationProperties
categories [roman, bilim, yazılım], currency TRY, free shipping from 500.00, delivery
↪ PT48H
```

Testi: store/StorePropertiesTest — boş ad, negatif ücret ve geçersiz e-posta ile uygulamanın başlamadığını doğrular.

! Uyarı

YAML'da tırnaksız 500.00 bir **ondalık sayıdır** ve 500.0 olarak okunur. Para gibi hassas değerleri tırnak içinde yazın: free-from: "500.00".

★ İpucu

spring-boot-configuration-processor, @ConfigurationProperties sınıflarınız için metadata üretir. IDE, application.yaml içinde bu ayarları otomatik tamamlar ve açıklamalarını gösterir. Kursun tüm modüllerinde açıktır.

3.3 @Value ile Tek Değer Enjeksiyonu

Amaç: @Value'nun nerede işe yaradığını ve neden az kullanılması gerektiğini görmek.

```
@Component
public class StoreInfo {

    private final String name;
    private final String phone;
    private final int maxBooksPerOrder;

    public StoreInfo(@Value("${bookstore.store.name}") String name,
        ↪ // required
        @Value("${bookstore.store.phone:+90 212 000 00 00}") String phone,
        ↪ // with default
        @Value("#{2 + 3}") int maxBooksPerOrder) {
        ↪ // SpEL expression
        this.name = name;
        this.phone = phone;
        this.maxBooksPerOrder = maxBooksPerOrder;
    }
}
```

`${anahtar:varsayılan}` bir varsayılan değer verir. `#{...}` ise bir SpEL ifadesidir. Varsayılanı olmayan eksik bir anahtar, uygulamanın başlamasını engeller.

Beklenen çıktı:

```
== 3.3 @Value
Kitapçı · +90 212 000 00 00 · max 5 kitap / books
```

Testi: legacy/StoreInfoTest

	@ConfigurationProperties	@Value
Birden çok ilişkili ayar	Evet, tek nesnede	Her biri ayrı ayrı
Doğrulama	@Validated ile	Yok
Relaxed binding	Tam	Sınırlı
IDE otomatik tamamlama	Evet (metadata)	Hayır
SpEL	Hayır	Evet

3.4 Profiller ve Profil Grupları

Amaç: Ortama göre ayar değiştirmek ve birden çok profili tek adla açmak.

application-dev.yaml, yalnızca dev profili açıkken yüklenir ve varsayılanları ezer:

```
# Lesson 3.4 – loaded only with the "dev" profile; overrides application.yaml
bookstore:
  store:
    name: Kitapçı (DEV)
    shipping:
      delivery-time: 1h
```

Aynı dosyada, --- ile ayrılmış ve yalnızca bir profilde geçerli olan bir belge de yazılabilir:

```
# Lesson 3.4 – a second YAML document, applied only when the "demo" profile is active
spring:
  config:
    activate:
      on-profile: demo
bookstore:
  store:
    banner: "DEMO – veriler gerçek değil / demo data"
```

spring.profiles.group.local: dev, demo tanımı sayesinde local profili, dev ve demo profillerini birlikte açar.

Çalıştırın:

```
./mvnw -pl modules/02-configuration/lesson -am spring-boot:run \
-Dspring-boot.run.arguments="--spring.profiles.active=local"
```

Beklenen çıktı:

```
== 3.4 Profiles
active profiles [local, dev, demo], banner: DEMO – veriler gerçek değil / demo data
```

Testi: profiles/ProfileGroupTest

3.5 spring.config.import

Amaç: Ayarları birden çok dosyaya bölmek ve makineye özel, git'e girmeyen bir dosyayı isteğe bağlı yüklemek.

application.yaml iki dosya içe aktarır. optional: öneki, dosya yoksa hata verilmemesini sağlar:

```
config:
  import:
    - optional:classpath:campaigns.yaml
    - optional:file:./bookstore-local.yaml      # your machine-only overrides
    ↪ (git-ignored)
```

İçe aktarılan campaigns.yaml:

```
# Lesson 3.5 – imported by application.yaml via spring.config.import
bookstore:
  campaigns:
    active:
      - code: OKULA-DONUS
        percent: 15
      - code: KITAP-FUARI
        percent: 25
```

Beklenen çıktı:

```
== 3.5 spring.config.import
OKULA-DONUS → %15
KITAP-FUARI → %25
```

Testi: imports/ConfigImportTest

★ **İpucu**

modules/02-configuration/lesson/bookstore-local.yaml dosyası oluşturup içine örneğin `bookstore.store.name: Benim Kitapçım` yazın ve uygulamayı tekrar çalıştırın. Bu dosya `.gitignore`'dadır. Şifre gibi kişisel ayarlar için idealdir.

3.6 Kendi Starter'inizi Yazmak

Amaç: Bir auto-configuration yazmak ve onu tek bir bağımlılıkla kullanılabilir bir starter'a dönüştürmek.

Bir starter iki parçadan oluşur:

Modül	İçerik
bookstore-greeting-autoconfigure	Kod, ayarlar ve koşullar
bookstore-greeting-spring-boot-starter	Kod yok. Yalnızca bağımlılıkları bir araya getirir.

Önce starter'ın ayarları:

```
@ConfigurationProperties("bookstore.greeting")
public record GreetingProperties(
    @DefaultValue("true") boolean enabled,
    @DefaultValue("Merhaba") String prefix,
    @DefaultValue("!") String suffix) {
}
```

Sonra koşullarla korunan auto-configuration:

```
@AutoConfiguration
@ConditionalOnClass(GreetingService.class) // only if the class is
↳ on the classpath
@ConditionalOnBooleanProperty(name = "bookstore.greeting.enabled", matchIfMissing = true)
@EnableConfigurationProperties(GreetingProperties.class)
public class GreetingAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean // back off if the
    ↳ application has its own
    GreetingService greetingService(GreetingProperties properties) {
        return name -> properties.prefix() + ", " + name + properties.suffix();
    }
}
```

Son olarak Boot'un onu bulması için sınıf adı imports dosyasına yazılır:

```
# Lesson 3.6 – Spring Boot loads every auto-configuration class listed here (one per  
↪ line).  
com.springbootedu.greeting.autoconfigure.GreetingAutoConfiguration
```

Ders uygulaması yalnızca starter bağımlılığını ekler ve bookstore.greeting.prefix: Hoş geldiniz ayarını yapar. Bean kendiliğinden gelir.

Beklenen çıktı:

```
== 3.6 Our own starter  
Hoş geldiniz, Ayşe!
```

Testleri: starter/.../GreetingAutoConfigurationTest her koşulu ayrı ayrı dener (varsayılan, ayarlar, geri çekilme, kapatma, eksik sınıf, imports dosyası). Ders tarafında starter/GreetingStarterUsageTest vardır.

! Önemli

İsimlendirme kuralı: resmî starter'lar spring-boot-starter-*, üçüncü taraf starter'lar *-spring-boot-starter biçimindedir. Kendi modüllerinizin adını spring-boot ile başlatmayın.

3.7 Conditions Report ile Hata Ayıklama

Amaç: Bir auto-configuration'ın neden çalıştığını veya çalışmadığını bulmak.

Uygulamayı - -debug ile başlattığınızda Boot, **conditions evaluation report**'u konsola basar. Aynı bilgiye kod içinden de erişilebilir:

```
@Component  
public class ConditionsExplainer {  
  
    private final ConfigurableListableBeanFactory beanFactory;  
  
    public ConditionsExplainer(ConfigurableListableBeanFactory beanFactory) {  
        this.beanFactory = beanFactory;  
    }  
  
    public String explain(String autoConfigurationSimpleName) {  
        Map<String, ConditionAndOutcomes> outcomes =  
  
↪ ConditionEvaluationReport.get(beanFactory).getConditionAndOutcomesBySource();
```

```
return outcomes.entrySet().stream()
    .filter(entry -> entry.getKey().endsWith(".") +
        ↳ autoConfigurationSimpleName))
    .findFirst()
    .map(entry -> (entry.getValue().isFullMatch() ? "MATCHED: " : "SKIPPED: ")
        + entry.getValue().stream()
            .map(outcome -> outcome.getOutcome().getMessage())
            .collect(Collectors.joining("; ")))
    .orElse("NOT A CANDIDATE: its module is not on the classpath");
}
```

Beklenen çıktı:

```
== 3.7 Conditions report
GreetingAutoConfiguration      MATCHED: @ConditionalOnClass found required class
↳ '...GreetingService'; @ConditionalOnBooleanProperty
↳ (bookstore.greeting.enabled=true) matched
MessageSourceAutoConfiguration SKIPPED: ResourceBundle did not find bundle with basename
↳ messages
DataSourceAutoConfiguration     NOT A CANDIDATE: its module is not on the classpath
```

Tam raporu görmek için:

```
./mvnw -pl modules/02-configuration/lesson -am spring-boot:run
↳ -Dspring-boot.run.arguments="--debug"
```

Testi: diagnostics/ConditionsExplainerTest

i Not

Sınıfın adı bilerek `AutoConfigurationReport` değildir. Boot, kendi `ConditionEvaluationReport` bean'ini tam olarak bu adla kaydeder. Aynı adı kullanmak bir bean çakışmasına yol açar.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Şifre, API anahtarı gibi sırları `application.yaml`'a yazıp git'e göndermeyin. Ortam değişkeni, `optional:file:` ile içe aktarılan yerel bir dosya veya bir secret yöneticisi kullanın.

- **Yapın:** İlişkili ayarları bir `@ConfigurationProperties` record'unda toplayın ve `@Validated` ile başlangıçta doğrulayın.
- **Yapmayın:** Aynı anahtarları onlarca `@Value` ile farklı sınıflara dağıtmayın.
- **Yapın:** Süreleri `Duration`, boyutları `DataSize` olarak bağlayın (2d, 10MB).
- **Yapmayın:** Ondalık para değerlerini YAML'da tırnaksız yazmayın.
- **Yapın:** Auto-configuration'larınızda kullanıcı bean'leri için `@ConditionalOnMissingBean` ile geri çekilin ve her koşulu `ApplicationContextRunner` ile test edin.
- **Yapmayın:** Auto-configuration sınıflarını component scanning'e bırakmayın. Yalnızca imports dosyası ile yüklensinler.
- **Yapın:** "Bu bean neden yok?" sorusunda önce `--debug` ile conditions report'a bakın.

5. Özet

- Environment, sıralı bir kaynak listesidir. Daha dışarıdaki ve daha özel olan kaynak kazanır.
- `@ConfigurationProperties` record'ları tipli, doğrulanmış ve IDE dostu ayarlar sağlar. `@Value` tekil değerler içindir.
- Profil dosyaları, profil grupları ve on-profile belgeleri ortama göre ayar değiştirir.
- `spring.config.import`, ayarları birden çok dosyaya böler. `optional`: eksik dosyaya tolerans gösterir.
- Auto-configuration, koşullarla korunan bir `@Configuration` sınıfıdır. Starter ise yalnızca bağımlılıkları toplar.
- Conditions report, auto-configuration kararlarını açıklar.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Boot — Externalized Configuration](#)
- [Spring Boot — Profiles](#)
- [Spring Boot — Auto-configuration](#)
- [Spring Boot — Creating Your Own Auto-configuration](#)
- [Spring Boot — Configuration Metadata](#)